



UNIVERSITÀ DI PISA
LAUREA MAGISTRALE IN INGEGNERIA ELETTRONICA

TRASFORMAZIONE DA COORDINATE
CARTESIANE A POLARI MEDIANTE CORDIC

CORSO DI PROGETTAZIONE DI SISTEMI MICROELETTRONICI
(MODULO DIGITALE)

Author(s)
Matteo Tonini Antonio Tuma

Anno Accademico 2025/2026

Indice

1	Specifiche del progetto	1
2	Modello high-level	4
3	Progetto RTL	6
3.1	Architettura generale del modulo CORDIC	6
3.2	Macchina a stati finiti	7
3.3	Datapath CORDIC	9
3.4	LUT degli angoli	9
3.5	Contatore delle iterazioni	10
3.6	Correzione del guadagno CORDIC	10
3.7	Gestione dei casi particolari e controllo di convergenza	10
3.8	Wrapper per FPGA	11
4	Specifiche di interfaccia e comportamento atteso	12
4.1	Interfaccia del modulo CORDIC	12
4.2	Formato fixed-point	12
4.3	Sequenza di utilizzo del modulo	13
4.4	Temporizzazione attesa	14
4.5	Comportamento del segnale valid	15
4.6	Casi principali di funzionamento	15
4.7	Gestione dei quadranti	15
4.8	Interfaccia del wrapper per FPGA	16
4.9	Utilizzo degli switch nel wrapper	16
4.10	Valori attesi nel wrapper	17
5	Verifica funzionale	18
5.1	Struttura del testbench	18
5.2	Test realizzati	19
5.3	Risultati complessivi della simulazione	19
5.4	Analisi di un caso nominale	20

5.5	Gestione del vettore nullo	21
5.6	Verifica del secondo impulso di start durante RUN	21
5.7	Discussione dei risultati	22
6	Risultati di implementazione su FPGA	23
6.1	Configurazione del progetto Vivado	23
6.2	Utilizzo delle risorse FPGA	24
6.3	Analisi temporale	25
6.4	Analisi di potenza	26
6.5	Considerazioni finali sull'implementazione	27
7	Conclusioni	28

CAPITOLO 1

Specifiche del progetto

L'obiettivo del progetto è la progettazione RTL e la verifica funzionale di un modulo hardware basato su algoritmo CORDIC per la trasformazione da coordinate cartesiane a coordinate polari. A partire da due coordinate cartesiane signed, indicate con x e y , il modulo calcola il modulo ρ e l'angolo di fase θ del vettore di ingresso.

La trasformazione matematica attesa è:

$$\rho = \sqrt{x^2 + y^2} \quad (1.1)$$

$$\theta = \text{atan2}(y, x) \quad (1.2)$$

L'implementazione è basata sull'algoritmo CORDIC in modalità *vectoring*. Tale algoritmo è particolarmente adatto alla realizzazione hardware, poiché permette di sostituire operazioni complesse, moltiplicazioni, divisioni e radici quadrate, con somme, sottrazioni, shift aritmetici e una piccola tabella di lookup.

Le equazioni iterative del CORDIC utilizzate nel progetto sono:

$$x_{i+1} = x_i - y_i d_i 2^{-i} \quad (1.3)$$

$$y_{i+1} = y_i + x_i d_i 2^{-i} \quad (1.4)$$

$$z_{i+1} = z_i - d_i \arctan(2^{-i}) \quad (1.5)$$

dove il parametro d_i viene scelto in funzione del segno di y_i , in modo da portare progressivamente la componente y verso zero. Al termine delle iterazioni, la componente x risulta proporzionale al modulo del vettore, mentre l'accumulatore angolare z contiene l'angolo del vettore.

Poiché le rotazioni CORDIC introducono un guadagno costante, il modulo finale viene corretto moltiplicando il valore finale di (x) per l'inverso del guadagno CORDIC:

$$K_N = \frac{1}{A_N} \quad (1.6)$$

Nel progetto sono state utilizzate $(N = 16)$ iterazioni. Per tale scelta il valore utilizzato è:

$$K_N \approx 0.607252935 \quad (1.7)$$

La costante corrispondente in formato fixed-point è:

$$K_{\text{fixed}} = 39797 \quad (1.8)$$

Le principali specifiche del progetto sono riportate in Tabella 1.1.

Parametro	Valore
Algoritmo	CORDIC in modalità vectoring
Trasformazione	Coordinate cartesiane in coordinate polari
Formato degli ingressi	Signed Q2.14
Larghezza degli ingressi	16 bit
Formato interno	Signed Q4.16
Larghezza interna	20 bit
Formato del modulo ρ	Signed Q4.16
Formato dell'angolo θ	Signed Q4.16, radianti
Numero di iterazioni	16
Contenuto della LUT	$\arctan(2^{-i})$, con $i = 0, \dots, 15$
Correzione del guadagno	Moltiplicazione per $K_N \approx 0.607252935$
Reset	Attivo alto
Segnale di uscita valido	Asserito quando il risultato è disponibile

Tabella 1.1: Specifiche principali del progetto.

Le coordinate di ingresso sono rappresentate in formato fixed-point signed Q2.14. Sebbene il modulo ρ sia, per definizione, una grandezza non negativa, nel progetto è stato rappresentato mediante il tipo `signed`. Questa scelta è stata adottata per mantenere uniforme il datapath fixed-point interno, che utilizza segnali signed per le operazioni CORDIC e per la moltiplicazione finale con la costante di correzione del guadagno. Pertanto, pur essendo codificato come signed Q4.16, il valore valido di ρ prodotto dal modulo risulta sempre non negativo.

Il valore reale associato a un campione di ingresso è dato da:

$$x_{\text{reale}} = \frac{x_{\text{intero}}}{2^{14}} \quad (1.9)$$

Il datapath interno e le uscite utilizzano invece il formato signed Q4.16. Pertanto, i valori reali delle uscite sono interpretati come:

$$\rho_{\text{reale}} = \frac{\rho_{\text{intero}}}{2^{16}} \quad (1.10)$$

$$\theta_{\text{reale}} = \frac{\theta_{\text{intero}}}{2^{16}} \quad (1.11)$$

Il progetto prevede anche la gestione di alcuni casi particolari. Nello specifico, il vettore nullo $(0,0)$ viene riconosciuto e produce in uscita $\rho = 0$, $\theta = 0$ e segnale `valid` attivo. Inoltre, poiché l'algoritmo CORDIC base converge direttamente nel semipiano destro, è stata introdotta una correzione di quadrante per i vettori con $(x < 0)$. In questo caso il vettore viene trasformato secondo la relazione $(x,y) \rightarrow (-x,-y)$, mentre all'angolo finale viene aggiunto un offset pari a $+\pi$ oppure $-\pi$, a seconda del segno originale della coordinata y .

Il modulo è controllato tramite una macchina a stati finiti. Gli ingressi vengono acquisiti quando il sistema si trova nello stato di attesa e il segnale `start` viene portato a livello alto. Eventuali impulsi di `start` applicati durante l'elaborazione vengono ignorati, in modo da evitare che il campione in fase di elaborazione venga sovrascritto prima della produzione del risultato.

È stato inoltre realizzato un wrapper di livello superiore per una possibile verifica su FPGA. Poiché le schede considerate dispongono di un numero limitato di switch e LED fisici, il wrapper utilizza alcuni vettori di test predefiniti e permette di visualizzare solo una porzione selezionabile delle uscite a 20 bit.

CAPITOLO 2

Modello high-level

Il progetto non richiede un modello high-level completo dell'intero datapath CORDIC, poiché la verifica funzionale è stata svolta direttamente sul modello RTL tramite testbench VHDL. Tuttavia, è stato utilizzato uno script Python come supporto alla progettazione per generare automaticamente la tabella di lookup contenente i valori di $\arctan(2^{-i})$, necessari all'algoritmo CORDIC.

La LUT viene utilizzata durante le iterazioni per aggiornare l'accumulatore angolare z . A ogni iterazione i , il valore letto dalla tabella corrisponde a:

$$\alpha_i = \arctan(2^{-i}) \quad (2.1)$$

con:

$$i = 0, 1, \dots, 15 \quad (2.2)$$

Poiché nel progetto sono state utilizzate 16 iterazioni, la LUT contiene 16 valori. Gli angoli sono rappresentati in radianti e codificati in formato fixed-point Q4.16, coerentemente con il formato utilizzato internamente per l'accumulatore angolare z . Il valore intero associato a ciascun angolo è quindi ottenuto mediante la relazione:

$$\alpha_{i,\text{fixed}} = \text{round}(\arctan(2^{-i}) \cdot 2^{16}) \quad (2.3)$$

Lo script Python consente di evitare errori manuali nella trascrizione dei coefficienti e rende la generazione della LUT ripetibile. Il file VHDL generato, denominato `atan_lut.vhd`, è stato incluso tra i sorgenti RTL del progetto.

Listing 2.1: Script Python per la generazione della LUT degli angoli $\arctan$.

```
1 import math
2
```

```

3 ANGLE_WIDTH = 20
4 FRAC_BITS = 16
5 ITERATIONS = 16
6
7 print("library_ieee;")
8 print("use_ieee.std_logic_1164.all;")
9 print("use_ieee.numeric_std.all;")
10 print("")
11 print("entity_atan_lut_is")
12 print("_port_")
13 print("_addr_:_in_unsigned(3_downto_0);")
14 print("_atan_value_:_out_signed(19_downto_0)")
15 print("_");
16 print("end_entity_atan_lut;")
17 print("")
18 print("architecture_rtl_of_atan_lut_is")
19 print("begin")
20 print("")
21 print("_process(addr)")
22 print("_begin")
23 print("_case_addr_is")
24
25 for i in range(ITERATIONS):
26     angle = math.atan(2 ** (-i))
27     fixed_value = round(angle * (2 ** FRAC_BITS))
28     binary_value = format(fixed_value, "020b")
29
30 print(f'_____when_{i:04b}"_=>')
31 print(f'_____atan_value_<=_"{binary_value}";')
32
33 print("_when_others_>")
34 print("_atan_value_<=_ (others_>_ '0');")
35 print("_end_case;")
36 print("_end_process;")
37 print("")
38 print("end_architecture_rtl;")

```

Il modello high-level è quindi limitato alla generazione dei coefficienti costanti utilizzati dal CORDIC. Questa scelta è coerente con l'obiettivo del progetto, che è principalmente la progettazione RTL del modulo hardware. Lo script Python non implementa direttamente il comportamento sequenziale del circuito, ma produce una parte necessaria e verificabile del design, cioè la LUT degli angoli.

CAPITOLO 3

Progetto RTL

Il progetto RTL è stato sviluppato con una struttura gerarchica. Il blocco principale è rappresentato dal modulo `CORDIC`, che implementa l'algoritmo CORDIC in modalità *vectoring*. Attorno a questo blocco sono stati utilizzati moduli ausiliari per la memorizzazione degli ingressi, il conteggio delle iterazioni e la lettura dei coefficienti angolari dalla LUT.

I principali file VHDL che compongono il progetto sono:

File	Funzione
<code>CORDIC.vhd</code>	Modulo principale CORDIC
<code>atan_lut.vhd</code>	LUT contenente i valori di $\arctan(2^{-i})$
<code>Counter.vhd</code>	Contatore delle iterazioni
<code>DFF_N.vhd</code>	Registro parametrico per segnali <code>std_logic_vector</code>
<code>DFF_N_Signed.vhd</code>	Registro parametrico per segnali <code>signed</code>
<code>ripple_carry_adder.vhd</code>	Sommatore ripple carry parametrico
<code>full_adder.vhd</code>	Cella elementare di somma a un bit
<code>CORDIC_WRAPPER.vhd</code>	Wrapper per il test su FPGA

Tabella 3.1: Organizzazione dei file RTL del progetto.

3.1 Architettura generale del modulo CORDIC

Il modulo `CORDIC` riceve in ingresso due coordinate cartesiane x e y , rappresentate in formato `signed Q2.14` su 16 bit. Gli ingressi vengono campionati in due registri dedicati, `x_in_reg` e `y_in_reg`, quando il segnale `start_load` è attivo.

Internamente, i valori vengono convertiti dal formato `Q2.14` al formato `Q4.16` mediante estensione di segno e shift a sinistra di due posizioni. Questa scelta permette di

aumentare la precisione durante le iterazioni e di avere un datapath interno uniforme a 20 bit.

L'architettura generale del modulo può essere schematizzata come in Figura 3.1.

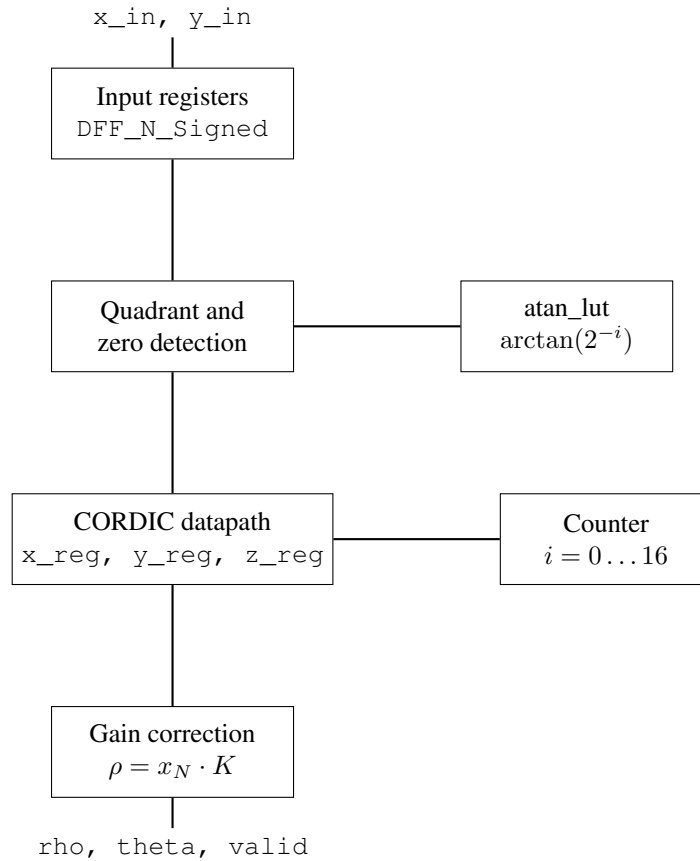


Figura 3.1: Schema funzionale ad alto livello del modulo *CORDIC*.

I registri principali del datapath sono:

- `x_reg`: contiene la componente x corrente del vettore;
- `y_reg`: contiene la componente y corrente del vettore;
- `z_reg`: contiene l'accumulatore angolare;
- `theta_offset`: contiene l'eventuale correzione di quadrante.

A ogni iterazione, i valori `x_reg`, `y_reg` e `z_reg` vengono aggiornati attraverso un processo combinatorio che calcola i segnali `x_next`, `y_next` e `z_next`. L'aggiornamento dipende dal segno della componente y . Se y è negativa, il datapath aggiorna i registri in modo da aumentare y verso zero; se invece y è positiva o nulla, il datapath aggiorna i registri in modo da diminuire y verso zero.

3.2 Macchina a stati finiti

Il controllo del modulo è realizzato mediante una macchina a stati finiti composta da quattro stati:

- IDLE;
- LOAD;
- RUN;
- DONE.

Il comportamento della FSM è riassunto in Tabella 3.2.

Stato	Funzione	Transizione principale
IDLE	Attesa del segnale <i>start</i>	Passa a LOAD se <i>start</i> = '1'
LOAD	Caricamento e preparazione degli ingressi	Passa a RUN o direttamente a DONE
RUN	Esecuzione delle iterazioni CORDIC	Rimane in RUN finché $i < 16$
DONE	Produzione di ρ , θ e <i>valid</i>	Torna a IDLE

Tabella 3.2: Stati principali della macchina a stati finiti.

Lo schema logico della FSM è riportato in Figura 3.2.

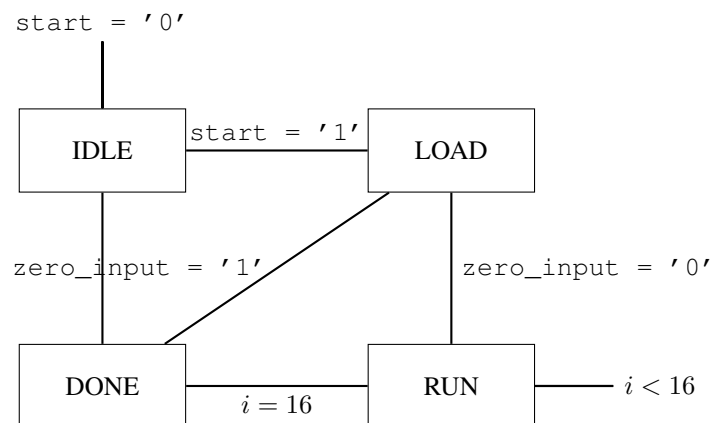


Figura 3.2: Schema della macchina a stati finiti del modulo CORDIC.

Nello stato IDLE, il modulo hardware attende l'arrivo di un impulso sul segnale *start*. Quando *start* è alto, viene generato il segnale *start_load*, che abilita il caricamento degli ingressi nei registri *x_in_reg* e *y_in_reg*.

Nello stato LOAD, il modulo hardware prepara i valori iniziali per le iterazioni. Se il vettore di ingresso è nullo, cioè $(0, 0)$, il modulo salta direttamente allo stato DONE. In caso contrario, i valori vengono caricati nei registri interni *x_reg* e *y_reg*. Se l'ingresso ha $x < 0$, il vettore viene trasformato in $(-x, -y)$, in modo da portarlo nel semipiano destro, e viene memorizzata una correzione angolare in *theta_offset*. Se $y \geq 0$, l'offset è pari a $+\pi$; se $y < 0$, l'offset è pari a $-\pi$.

Nello stato RUN, il modulo hardware esegue le iterazioni CORDIC. Il contatore fornisce l'indice i , utilizzato sia per effettuare gli shift aritmetici sia per indirizzare la LUT degli angoli. Quando $i = 16$, le iterazioni sono terminate e la FSM passa allo stato DONE.

Nello stato DONE, il modulo hardware calcola il valore finale di ρ , correggendo il guadagno CORDIC, e produce il valore finale di θ , sommando l'eventuale offset di

quadrante. Il segnale `valid` viene generato in questa fase per indicare la disponibilità del risultato.

3.3 Datapath CORDIC

Il datapath CORDIC è basato su tre registri principali, `x_reg`, `y_reg` e `z_reg`. A ogni ciclo di clock nello stato RUN, il valore successivo dei registri viene calcolato in modo combinatorio.

Il processo combinatorio distingue due casi:

- se `y_reg` è minore di zero, la componente y viene aumentata verso zero;
- se `y_reg` è maggiore o uguale a zero, la componente y viene diminuita verso zero.

Nel primo caso, il datapath esegue:

$$x_{i+1} = x_i - (y_i \gg i) \quad (3.1)$$

$$y_{i+1} = y_i + (x_i \gg i) \quad (3.2)$$

$$z_{i+1} = z_i - \arctan(2^{-i}) \quad (3.3)$$

Nel secondo caso, il datapath esegue:

$$x_{i+1} = x_i + (y_i \gg i) \quad (3.4)$$

$$y_{i+1} = y_i - (x_i \gg i) \quad (3.5)$$

$$z_{i+1} = z_i + \arctan(2^{-i}) \quad (3.6)$$

Lo shift viene implementato tramite la funzione `shift_right` della libreria `numeric_std`. Poiché i segnali sono di tipo `signed`, lo shift è aritmetico e conserva correttamente il segno del dato.

3.4 LUT degli angoli

La LUT degli angoli è implementata nel file `atan_lut.vhd`. Essa contiene i 16 valori:

$$\arctan(2^{-i}), \quad i = 0, 1, \dots, 15 \quad (3.7)$$

I valori sono rappresentati in formato signed Q4.16 su 20 bit. Il contatore genera l'indice dell'iterazione corrente e i quattro bit meno significativi del contatore vengono utilizzati come indirizzo della LUT. La LUT restituisce il valore `atan_i`, che viene sommato o sottratto all'accumulatore angolare `z_reg`.

3.5 Contatore delle iterazioni

Il contatore è implementato nel file `Counter.vhd`. La sua struttura è gerarchica e utilizza:

- un registro parametrico `DFF_N`;
- un sommatore `ripple_carry_adder`;
- più celle `full_adder`.

Nel progetto il contatore ha larghezza pari a 5 bit, poiché deve rappresentare anche il valore 16, utilizzato per riconoscere la fine delle iterazioni. Il segnale di incremento è pari a uno e il contatore viene abilitato nello stato `RUN` finché $i < 16$.

Il reset del contatore è attivo alto e viene generato quando il reset globale è attivo oppure quando la FSM si trova nello stato `LOAD`. In questo modo ogni nuova elaborazione inizia sempre dall'iterazione $i = 0$.

3.6 Correzione del guadagno CORDIC

Al termine delle iterazioni, la componente `x_reg` non coincide direttamente con il modulo del vettore di ingresso, ma risulta moltiplicata per il guadagno CORDIC. Per compensare tale effetto, il progetto utilizza la costante:

$$K_{\text{CORDIC}} = 0.607252935 \quad (3.8)$$

Nel codice RTL questa costante è rappresentata in formato Q1.16 mediante il valore intero:

$$K_{\text{fixed}} = 39797 \quad (3.9)$$

Il prodotto tra `x_reg` e `K_CORDIC` genera il segnale `rho_mult`. Poiché `x_reg` è in formato Q4.16 e `K_CORDIC` è in formato Q1.16, il prodotto è in formato Q5.32. Per riportare il risultato al formato Q4.16, il prodotto viene shiftato a destra di 16 bit e ridimensionato a 20 bit.

3.7 Gestione dei casi particolari e controllo di convergenza

Il progetto gestisce esplicitamente il vettore nullo. Se gli ingressi campionati sono entrambi pari a zero, il segnale `zero_input` viene posto a uno. In questo caso il modulo non esegue le iterazioni CORDIC e passa direttamente allo stato `DONE`, producendo $\rho = 0$ e $\theta = 0$.

Per i vettori con $x < 0$, il modulo hardware applica una correzione di quadrante. Poiché il CORDIC in modalità vectoring converge direttamente nel semipiano destro, il vettore viene trasformato in $(-x, -y)$. Successivamente, l'angolo viene corretto aggiungendo $+\pi$ o $-\pi$, in base al segno originario della coordinata y .

Infine, è presente un controllo di convergenza basato sul valore finale della componente y . Il segnale `convergence_ok` viene posto a uno quando il valore assoluto di `y_reg` è minore o uguale a una soglia pari a 4 LSB nel formato interno Q4.16. Questo

controllo permette di verificare che, al termine delle iterazioni, la componente y sia stata effettivamente portata vicino a zero.

Nel formato Q4.16, un LSB corrisponde a:

$$1 \text{ LSB} = 2^{-16} = \frac{1}{65536} \approx 1.5259 \cdot 10^{-5} \quad (3.10)$$

Pertanto, una soglia pari a 4 LSB equivale a:

$$4 \text{ LSB} = 4 \cdot 2^{-16} = \frac{4}{65536} \approx 6.1035 \cdot 10^{-5} \quad (3.11)$$

Questo significa che il controllo di convergenza considera accettabile un residuo sulla componente y inferiore o uguale a circa $6.1 \cdot 10^{-5}$, espresso nel valore reale associato al formato fixed-point interno. A livello angolare, per piccoli errori, il residuo può essere approssimato come:

$$\Delta\theta \approx \frac{y_{\text{residuo}}}{\rho} \quad (3.12)$$

Di conseguenza, per un vettore con modulo circa unitario, la soglia di 4 LSB corrisponde a un errore angolare indicativo pari a:

$$\Delta\theta \approx 6.1035 \cdot 10^{-5} \text{ rad} \approx 0.0035^\circ \quad (3.13)$$

Tuttavia, l'errore angolare effettivo dipende dal modulo del vettore: a parità di residuo su y , l'errore angolare risulta maggiore per vettori con modulo più piccolo.

3.8 Wrapper per FPGA

Oltre al modulo principale CORDIC, è stato realizzato il modulo CORDIC_WRAPPER, pensato per una possibile verifica su FPGA. Il wrapper collega il core CORDIC ai segnali fisici della scheda.

Il pulsante `btn_start` viene trasformato in un impulso di un solo ciclo di clock tramite un registro `start_reg`. Questo evita che una pressione prolungata del pulsante venga interpretata come più richieste di avvio. Il segnale `valid` prodotto dal core viene invece memorizzato nel segnale `valid_latched`, in modo da renderlo visibile sui LED anche se `valid` rimane alto per un solo ciclo di clock.

La selezione dei test e della visualizzazione avviene tramite gli switch della scheda. In particolare, `sw(0)` seleziona il vettore di test, `sw(1)` seleziona se visualizzare ρ oppure θ , `sw(2)` seleziona quale porzione dei 20 bit mostrare sui LED, mentre `sw(3)` permette di visualizzare il valore memorizzato di `valid`.

La struttura del wrapper permette quindi di verificare il funzionamento del core anche con un numero ridotto di ingressi e uscite fisiche disponibili sulla scheda.

Specifiche di interfaccia e comportamento atteso

In questo capitolo verrà illustrato il modo in cui il modulo `CORDIC` deve essere utilizzato, specificando il significato dei segnali di ingresso e di uscita, il formato fixed-point dei dati, la sequenza temporale di utilizzo e il comportamento atteso nei principali casi di funzionamento.

4.1 Interfaccia del modulo `CORDIC`

Il modulo principale del progetto è `CORDIC`. Esso riceve in ingresso due coordinate cartesiane x e y , e produce in uscita il modulo ρ e l'angolo θ .

L'interfaccia del modulo è riportata in Tabella 4.1.

I parametri generici utilizzati nella configurazione del progetto sono riportati in Tabella 4.2.

4.2 Formato fixed-point

Gli ingressi `x_in` e `y_in` sono rappresentati in formato signed Q2.14. Questo significa che il valore reale corrispondente a un valore intero di ingresso è:

$$x_{\text{reale}} = \frac{x_{\text{intero}}}{2^{14}} \quad (4.1)$$

$$y_{\text{reale}} = \frac{y_{\text{intero}}}{2^{14}} \quad (4.2)$$

Ad esempio, il valore reale (1.0) viene rappresentato in ingresso dal valore intero:

$$1.0 \cdot 2^{14} = 16384 \quad (4.3)$$

Segnale	Direzione	Tipo	Descrizione
clk	Ingresso	std_logic	Clock del sistema. Le operazioni sequenziali avvengono sul fronte di salita.
reset	Ingresso	std_logic	Reset asincrono attivo alto. Porta il modulo nello stato iniziale.
start	Ingresso	std_logic	Segnale di avvio del calcolo. Deve essere portato alto quando il modulo è in stato IDLE.
x_in	Ingresso	signed(DATA_IN-1 downto 0)	Coordinata cartesiana x , in formato signed Q2.14.
y_in	Ingresso	signed(DATA_IN-1 downto 0)	Coordinata cartesiana y , in formato signed Q2.14.
rho	Uscita	signed(DATA_INTERNAL-1 downto 0)	Modulo del vettore, in formato signed Q4.16. Il valore valido è sempre non negativo.
theta	Uscita	signed(ANGLE_WIDTH-1 downto 0)	Angolo del vettore, in radianti, rappresentato in formato signed Q4.16.
valid	Uscita	std_logic	Segnale che indica la disponibilità del risultato e la corretta convergenza del calcolo.

Tabella 4.1: *Interfaccia del modulo CORDIC.*

Parametro	Valore	Descrizione
DATA_IN	16	Larghezza degli ingressi x e y
DATA_INTERNAL	20	Larghezza del datapath interno e di ρ
ANGLE_WIDTH	20	Larghezza dell'accumulatore angolare e di θ
ITERATIONS	16	Numero di iterazioni CORDIC

Tabella 4.2: *Parametri generici del modulo CORDIC.*

Le uscite `rho` e `theta` sono invece rappresentate in formato signed Q4.16. I corrispondenti valori reali sono quindi:

$$\rho_{\text{reale}} = \frac{\rho_{\text{intero}}}{2^{16}} \quad (4.4)$$

$$\theta_{\text{reale}} = \frac{\theta_{\text{intero}}}{2^{16}} \quad (4.5)$$

Pertanto, il valore reale 1.0 viene rappresentato in uscita dal valore intero:

$$1.0 \cdot 2^{16} = 65536 \quad (4.6)$$

Analogamente, l'angolo π in formato Q4.16 è rappresentato approssimativamente dal valore intero:

$$\pi \cdot 2^{16} \approx 205887 \quad (4.7)$$

4.3 Sequenza di utilizzo del modulo

Il modulo deve essere utilizzato basandosi sul segnale `start` e sul segnale `valid`.

La sequenza corretta è la seguente:

1. portare eventualmente `reset` a uno per inizializzare il modulo;
2. riportare `reset` a zero;
3. applicare i valori desiderati agli ingressi `x_in` e `y_in`;
4. mantenere gli ingressi stabili e portare `start` a uno per almeno un fronte di salita del clock;
5. riportare `start` a zero;
6. attendere che il segnale `valid` venga portato a uno;
7. leggere i valori di `rho` e `theta`.

Gli ingressi vengono campionati quando il modulo si trova nello stato `IDLE` e il segnale `start` è alto. Dopo il campionamento, il modulo passa allo stato `LOAD` e poi allo stato `RUN`, in cui vengono eseguite le iterazioni CORDIC.

Durante l'esecuzione del calcolo non è necessario mantenere stabili `x_in` e `y_in`, poiché i valori sono già stati memorizzati nei registri di ingresso. Tuttavia, un nuovo comando di `start` deve essere applicato solo dopo la conclusione del calcolo precedente. Eventuali impulsi di `start` applicati mentre il modulo non si trova nello stato `IDLE` vengono ignorati.

4.4 Temporizzazione attesa

Nel caso generale, cioè per un vettore di ingresso diverso da $(0, 0)$, il modulo esegue 16 iterazioni CORDIC. Considerando anche gli stati di caricamento e completamento, il risultato viene prodotto dopo circa 19 cicli di clock rispetto al fronte di salita in cui viene acquisito il segnale `start`.

La sequenza temporale può essere riassunta come in Tabella 4.3.

Ciclo	Stato	Descrizione
0	IDLE	<code>start</code> viene campionato e gli ingressi vengono registrati
1	LOAD	Preparazione dei dati interni e gestione del quadrante
2–17	RUN	Esecuzione delle 16 iterazioni CORDIC
18	RUN	Riconoscimento della fine delle iterazioni
19	DONE	Aggiornamento di ρ , θ e generazione di <code>valid</code>

Tabella 4.3: Temporizzazione indicativa del modulo per un vettore non nullo.

Il segnale `valid` rimane alto per un solo ciclo di clock. Per questo motivo, l'utilizzatore del modulo deve campionare `rho` e `theta` nel ciclo in cui `valid` è pari a uno.

Le uscite `rho` e `theta` non vengono azzerate automaticamente nello stato `IDLE`; esse mantengono l'ultimo risultato calcolato fino a una nuova elaborazione o fino a un reset del modulo.

4.5 Comportamento del segnale valid

Il segnale `valid` indica che il risultato è disponibile e che il controllo di convergenza ha avuto esito positivo. Nel progetto, la convergenza viene verificata controllando che il valore assoluto della componente residua `y_reg` sia minore o uguale a 4 LSB nel formato interno Q4.16.

In termini reali, tale soglia vale:

$$4 \cdot 2^{-16} = \frac{4}{65536} \approx 6.1035 \cdot 10^{-5} \quad (4.8)$$

Se il residuo finale sulla componente y rispetta questa soglia, `valid` viene portato a uno nello stato `DONE`. In caso contrario, le uscite vengono comunque aggiornate, ma `valid` resta a zero e il risultato non deve essere considerato valido dall'utilizzatore esterno.

4.6 Casi principali di funzionamento

La Tabella 4.4 riporta alcuni casi principali di funzionamento, con valori attesi espressi sia in forma reale sia in forma fixed-point. I valori fixed-point sono approssimati, poiché il risultato effettivo può differire di pochi LSB a causa delle approssimazioni introdotte dall'algoritmo CORDIC e dalla rappresentazione fixed-point.

x	y	ρ atteso	θ atteso	Note
1	0	$1 \approx 65536$	$0 \approx 0$	Asse x positivo
1	1	$\sqrt{2} \approx 92781$	$\pi/4 \approx 51472$	Primo quadrante
1	-1	$\sqrt{2} \approx 92781$	$-\pi/4 \approx -51472$	Quarto quadrante
-1	1	$\sqrt{2} \approx 92781$	$3\pi/4 \approx 154415$	Secondo quadrante
-1	-1	$\sqrt{2} \approx 92781$	$-3\pi/4 \approx -154415$	Terzo quadrante
0	0	0	0	Vettore nullo

Tabella 4.4: Esempi di valori attesi del modulo CORDIC.

Nel caso del vettore nullo $(0,0)$, l'angolo matematicamente non sarebbe definito. Nel progetto, per rendere deterministico il comportamento del circuito, è stata scelta l'uscita convenzionale $\rho = 0$ e $\theta = 0$, con `valid` attivo.

4.7 Gestione dei quadranti

L'algoritmo CORDIC in modalità *vectoring* converge direttamente nel semipiano destro. Per questo motivo, il modulo introduce una correzione di quadrante quando la coordinata di ingresso x è negativa.

Se $x \geq 0$, il vettore viene elaborato direttamente e l'offset angolare è nullo:

$$\theta_{\text{offset}} = 0 \quad (4.9)$$

Se invece $x < 0$, il vettore viene trasformato secondo:

$$(x, y) \rightarrow (-x, -y) \quad (4.10)$$

e viene poi aggiunto un offset angolare. In particolare:

$$\theta_{\text{offset}} = \begin{cases} +\pi, & \text{se } y \geq 0 \\ -\pi, & \text{se } y < 0 \end{cases} \quad (4.11)$$

L'angolo finale prodotto dal modulo è quindi:

$$\theta = z_{\text{finale}} + \theta_{\text{offset}} \quad (4.12)$$

dove (z_{finale}) è l'angolo accumulato durante le iterazioni CORDIC.

4.8 Interfaccia del wrapper per FPGA

Il progetto include anche il modulo `CORDIC_WRAPPER`, utilizzato per collegare il core `CORDIC` ai segnali fisici disponibili sulla scheda FPGA. Il wrapper utilizza due pulsanti, quattro switch e quattro LED.

L'interfaccia del wrapper è riportata in Tabella 4.5.

Segnale	Direzione	Descrizione
<code>clk</code>	Ingresso	Clock della scheda
<code>btn_reset</code>	Ingresso	Reset del sistema, attivo alto
<code>btn_start</code>	Ingresso	Pulsante di avvio del calcolo
<code>sw(3 downto 0)</code>	Ingresso	Switch di selezione test e visualizzazione
<code>led_out(3 downto 0)</code>	Uscita	LED utilizzati per visualizzare il risultato selezionato

Tabella 4.5: *Interfaccia del modulo `CORDIC_WRAPPER`.*

Il pulsante `btn_start` viene convertito internamente in un impulso di un solo ciclo di clock, chiamato `start_pulse`. In questo modo, anche se il pulsante viene mantenuto premuto per più cicli di clock, il core riceve un solo comando di avvio.

Il segnale `valid` prodotto dal core dura un solo ciclo di clock. Per renderlo osservabile sui LED, il wrapper lo memorizza nel segnale `valid_latched`, che rimane alto fino a un nuovo reset oppure fino all'avvio di una nuova elaborazione.

4.9 Utilizzo degli switch nel wrapper

Gli switch della scheda vengono utilizzati per selezionare il vettore di test e la porzione di risultato da visualizzare sui LED.

Switch	Valore	Funzione
<code>sw(0)</code>	0	Seleziona il test $x = -1, y = 0$
<code>sw(0)</code>	1	Seleziona il test $x = 1, y = 1$
<code>sw(1)</code>	0	Seleziona l'uscita ρ
<code>sw(1)</code>	1	Seleziona l'uscita θ
<code>sw(2)</code>	0	Visualizza i bit 19 downto 16 dell'uscita selezionata
<code>sw(2)</code>	1	Visualizza i bit 15 downto 12 dell'uscita selezionata
<code>sw(3)</code>	0	Visualizza la porzione selezionata di ρ o θ
<code>sw(3)</code>	1	Visualizza <code>valid_latched</code> su <code>led_out(0)</code>

Tabella 4.6: *Funzione degli switch nel wrapper FPGA.*

Quando $sw(3) = 1$, i LED mostrano:

$$led_out = "000" \text{ valid_latched} \quad (4.13)$$

quindi il LED meno significativo indica se il risultato è stato prodotto correttamente.

Quando invece $sw(3) = 0$, i LED mostrano una porzione dell'uscita selezionata. Poiché ρ e θ sono segnali a 20 bit, ma la scheda dispone solo di 4 LED, il wrapper permette di selezionare solo quattro bit alla volta.

4.10 Valori attesi nel wrapper

Nel wrapper sono presenti due vettori di test predefiniti. I valori attesi sono riportati in Tabella 4.7.

$sw(0)$	Ingresso selezionato	ρ atteso	θ atteso
0	$x = -1, y = 0$	$\rho \approx 1$	$\theta \approx \pi$
1	$x = 1, y = 1$	$\rho \approx \sqrt{2}$	$\theta \approx \pi/4$

Tabella 4.7: Vettori di test disponibili nel wrapper.

Per utilizzare il wrapper sulla scheda, la procedura è la seguente:

1. selezionare il test desiderato con $sw(0)$;
2. premere `btn_start` per avviare il calcolo;
3. selezionare $sw(3) = 1$ per verificare che `valid_latched` sia alto;
4. riportare $sw(3) = 0$;
5. usare $sw(1)$ per scegliere tra ρ e θ ;
6. usare $sw(2)$ per selezionare la porzione di bit da osservare sui LED.

Questa modalità di visualizzazione non permette di leggere direttamente l'intero risultato a 20 bit, ma consente comunque di verificare il comportamento del core su alcuni casi significativi, utilizzando soltanto le risorse fisiche disponibili sulla scheda.

CAPITOLO 5

Verifica funzionale

La verifica funzionale del modulo `CORDIC` è stata effettuata mediante simulazione in ModelSim, utilizzando un apposito testbench scritto in VHDL. L'obiettivo della verifica è stato controllare il corretto funzionamento del core nei casi nominali, nei casi di correzione di quadrante, nel caso del vettore nullo e nella gestione di impulsi di `start` applicati durante l'elaborazione.

5.1 Struttura del testbench

Il testbench, implementato nel file `CORDIC_tb.vhd`, istanzia il modulo `CORDIC` e genera i segnali necessari alla simulazione:

- clock di sistema con periodo pari a 8 ns;
- reset iniziale;
- segnali di ingresso `tb_x_in` e `tb_y_in`;
- segnale di avvio `tb_start`;
- monitoraggio delle uscite `tb_rho`, `tb_theta` e `tb_valid`.

Per semplificare l'applicazione degli stimoli è stata definita nel testbench una procedura `apply_input`, che riceve in ingresso due valori interi, li converte nel formato signed Q2.14 e genera un impulso di `start` della durata di un ciclo di clock. Successivamente la procedura attende la conclusione dell'elaborazione e lascia una breve pausa prima dell'applicazione del test successivo.

Questa organizzazione ha permesso di rendere il testbench più leggibile e di eseguire in modo ordinato una sequenza di test.

5.2 Test realizzati

La verifica è stata condotta mediante nove casi di test, scelti per coprire le principali condizioni operative del sistema. I test comprendono vettori nel primo, secondo, terzo e quarto quadrante, il vettore nullo, valori generici non simmetrici e il caso di un secondo impulso di `start` applicato durante lo stato RUN.

In Tabella 5.1 sono riportati i casi di test utilizzati.

Test	Ingresso	Valori Q2.14	Risultato atteso	Scopo
1	$x = 1; y = 0$	(16384; 0)	$\rho \approx 1; \theta \approx 0$	Asse (x) positivo
2	$x = 1; y = 1$	(16384; 16384)	$\rho \approx \sqrt{2}; \theta \approx \pi/4$	Primo quadrante
3	$x = 1; y = -1$	(16384; -16384)	$\rho \approx \sqrt{2}; \theta \approx -\pi/4$	Quarto quadrante
4	$x = -1; y = 1$	(-16384; 16384)	$\rho \approx \sqrt{2}; \theta \approx 3\pi/4$	Correzione di quadrante
5	$x = -1; y = -1$	(-16384; -16384)	$\rho \approx \sqrt{2}; \theta \approx -3\pi/4$	Correzione di quadrante
6	$x = 0; y = 0$	(0; 0)	$\rho = 0; \theta = 0$	Vettore nullo
7	$x = 0.5; y = 1$	(8192; 16384)	$\rho \approx 1.118; \theta \approx \arctan(2)$	Caso generico
8	$x = 1; y = -0.5$	(16384; -8192)	$\rho \approx 1.118; \theta \approx \arctan(-0.5)$	Caso generico
9	doppio <code>start</code> durante RUN	variabile	secondo <code>start</code> ignorato	Verifica del controllo

Tabella 5.1: Casi di test utilizzati per la verifica funzionale.

5.3 Risultati complessivi della simulazione

La Figura 5.1 mostra la waveform complessiva della simulazione, comprendente l'intera sequenza dei casi di test. Dalla figura si osserva che per ciascun vettore di ingresso viene generato un impulso di `start`, la macchina a stati attraversa le fasi di elaborazione e il segnale `valid` viene portato a livello alto quando il risultato è disponibile.

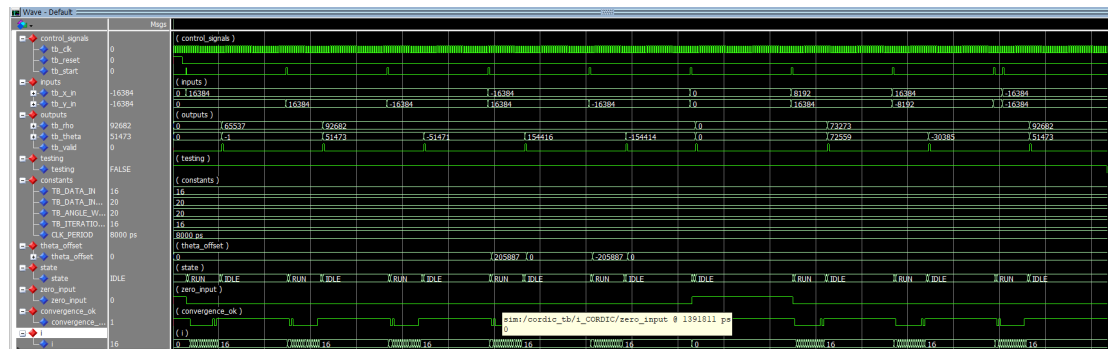


Figura 5.1: Waveform complessiva della simulazione del testbench, contenente l'esecuzione dei nove casi di test.

Dalla simulazione complessiva si osserva inoltre che:

- il contatore delle iterazioni raggiunge correttamente il valore 16 nei casi normali;

- il segnale `theta_offset` viene attivato soltanto nei casi con $x < 0$, come previsto dalla correzione di quadrante;
- il segnale `zero_input` viene attivato soltanto nel caso del vettore nullo;
- il segnale `convergence_ok` viene asserito al termine delle elaborazioni corrette.

5.4 Analisi di un caso nominale

La Figura 5.2 riporta uno zoom relativo al caso $x = 1, y = 1$, corrispondente a un vettore nel primo quadrante.

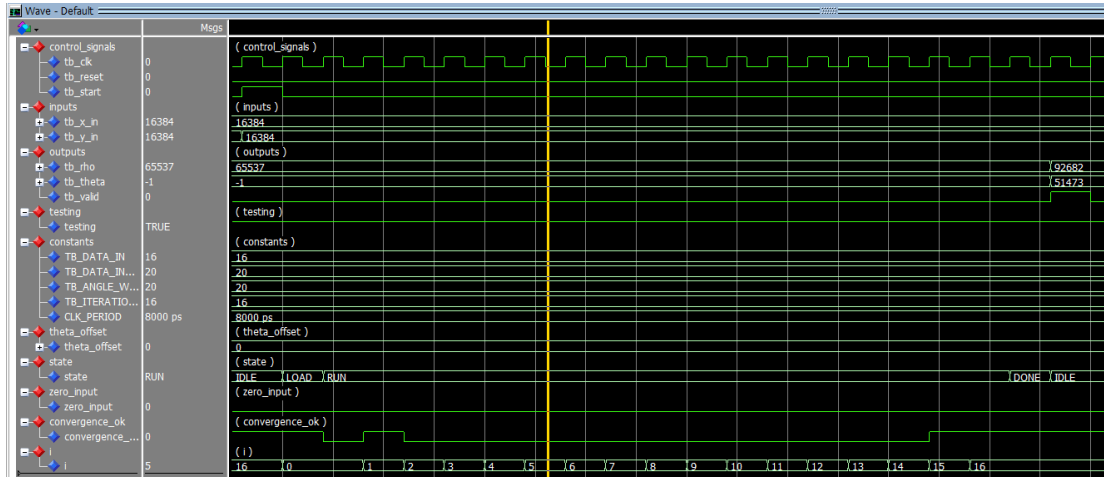


Figura 5.2: Zoom della simulazione per il caso $x = 1, y = 1$.

In questo caso il comportamento osservato è coerente con le aspettative:

- la FSM attraversa la sequenza $IDLE \rightarrow LOAD \rightarrow RUN \rightarrow DONE \rightarrow IDLE$;
- il contatore i avanza da 0 a 16;
- il valore finale del modulo è circa $\rho = 92682$, corrispondente a circa $92682/2^{16} \approx 1.4142$;
- il valore finale dell'angolo è circa $\theta = 51473$, corrispondente a circa $51473/2^{16} \approx 0.7854$ rad.

Tali valori sono coerenti con i risultati teorici attesi:

$$\rho = \sqrt{1^2 + 1^2} = \sqrt{2} \approx 1.4142 \quad (5.1)$$

$$\theta = \arctan\left(\frac{1}{1}\right) = \frac{\pi}{4} \approx 0.7854 \text{ rad} \quad (5.2)$$

La differenza tra valore teorico e valore simulato è limitata a pochi LSB ed è dovuta alla quantizzazione fixed-point e all'approssimazione introdotta dal numero finito di iterazioni.

5.5 Gestione del vettore nullo

La Figura 5.3 mostra il comportamento del modulo nel caso del vettore nullo, cioè per $x = 0$ e $y = 0$.

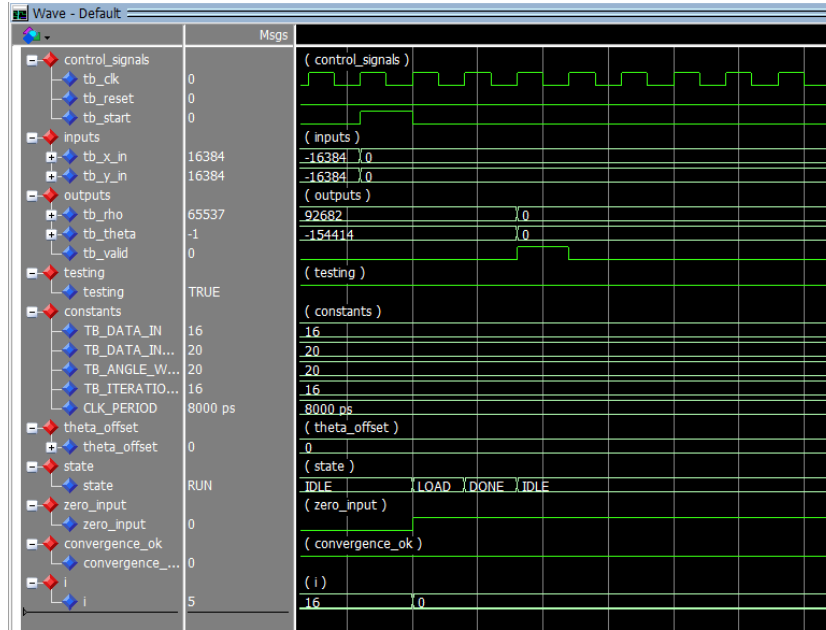


Figura 5.3: Zoom della simulazione per il caso del vettore nullo $(0, 0)$.

In questo caso il segnale `zero_input` viene posto a livello alto durante lo stato `LOAD`. Il modulo non entra nella normale sequenza di iterazioni, ma passa direttamente allo stato `DONE`, producendo:

$$\rho = 0 \quad (5.3)$$

$$\theta = 0 \quad (5.4)$$

Questo comportamento conferma la corretta gestione del caso particolare, già prevista nella fase di progetto.

5.6 Verifica del secondo impulso di start durante RUN

La Figura 5.4 mostra il caso in cui viene applicato un secondo impulso di `start` mentre il modulo sta ancora elaborando un campione precedente, cioè mentre la FSM si trova nello stato `RUN`.

Questa prova è stata inserita per verificare che il modulo non acquisisca nuovi ingressi durante un'elaborazione già in corso. Dalla waveform si osserva che il secondo impulso di `start` non provoca un nuovo caricamento dei registri di ingresso e non altera il risultato finale. Il valore prodotto in uscita corrisponde infatti al primo vettore applicato, mentre il secondo impulso viene correttamente ignorato.

Questo comportamento è coerente con la logica della FSM, che abilita il caricamento degli ingressi soltanto nello stato `IDLE`.

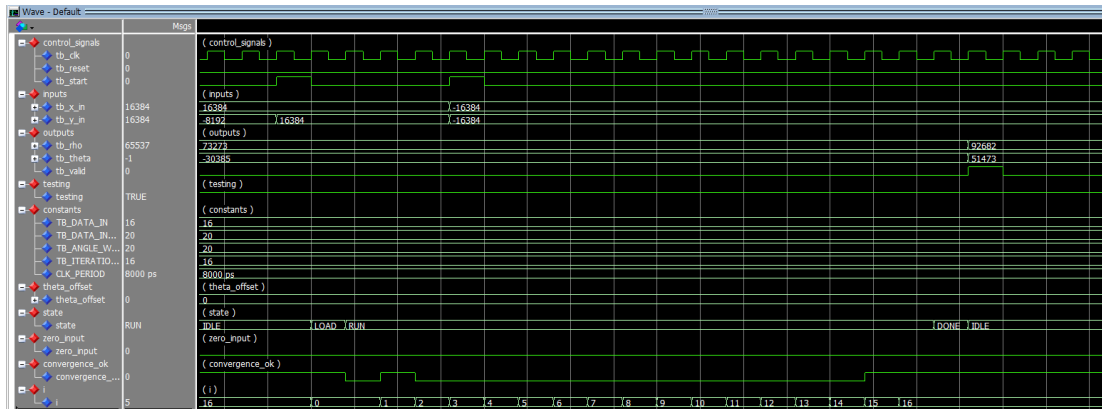


Figura 5.4: Zoom della simulazione per il caso di un secondo impulso di *start* applicato durante lo stato *RUN*.

5.7 Discussione dei risultati

I risultati ottenuti in simulazione confermano il corretto funzionamento del progetto RTL. In particolare:

- il modulo produce valori di ρ e θ coerenti con i risultati teorici attesi;
- la correzione di quadrante consente di trattare correttamente gli ingressi con $x < 0$;
- il vettore nullo viene riconosciuto e gestito senza eseguire iterazioni inutili;
- il segnale *valid* viene generato alla fine dell'elaborazione, come previsto;
- il modulo ignora impulsi di *start* applicati durante l'elaborazione, evitando sovrascritture del campione in corso.

Sono stati osservati soltanto piccoli scostamenti di pochi LSB rispetto ai valori ideali. Tali differenze sono compatibili con la rappresentazione fixed-point adottata e con l'approssimazione numerica introdotta dall'algoritmo CORDIC a 16 iterazioni.

La verifica funzionale può quindi considerarsi soddisfacente e conferma la correttezza dell'implementazione RTL sviluppata.

CAPITOLO 6

Risultati di implementazione su FPGA

In questo capitolo sono riportati i risultati ottenuti dall'implementazione del progetto su FPGA mediante Vivado. L'obiettivo di questa fase è stato verificare la sintetizzabilità del modulo, analizzare l'utilizzo delle risorse hardware, controllare il rispetto dei vincoli temporali e stimare la potenza assorbita dal circuito.

6.1 Configurazione del progetto Vivado

Il progetto è stato implementato utilizzando Vivado, impostando come modulo di livello superiore il wrapper `CORDIC_WRAPPER`. Tale modulo collega il core `CORDIC` ai segnali fisici della scheda.

La configurazione principale del progetto è riportata in Figura 6.1.

Settings	Edit
Project name:	CORDIC
Project location:	C:/PSM_digital_projects/home_project/vivado
Product family:	Zynq-7000
Project part:	xc7z010clg400-1
Top module name:	CORDIC_WRAPPER
Target language:	VHDL
Simulator language:	VHDL
Target Simulator:	Vivado Simulator

Figura 6.1: Configurazione principale del progetto Vivado.

Il dispositivo selezionato appartiene alla famiglia *Zynq-7000*. In particolare, il part number utilizzato è:

$$\text{xc7z010clg400-1} \quad (6.1)$$

Il linguaggio di progetto e di simulazione è VHDL, coerentemente con tutti i file sorgente sviluppati.

I principali parametri del progetto Vivado vengono riportati anche in Tabella 6.1.

Parametro	Valore
Nome progetto	CORDIC
Famiglia FPGA	Zynq-7000
Part number	xc7z010clg400-1
Top module	CORDIC_WRAPPER
Linguaggio target	VHDL
Simulatore	Vivado Simulator

Tabella 6.1: Impostazioni principali del progetto Vivado.

6.2 Utilizzo delle risorse FPGA

La Figura 6.2 mostra il report di utilizzo delle risorse dopo l'implementazione.

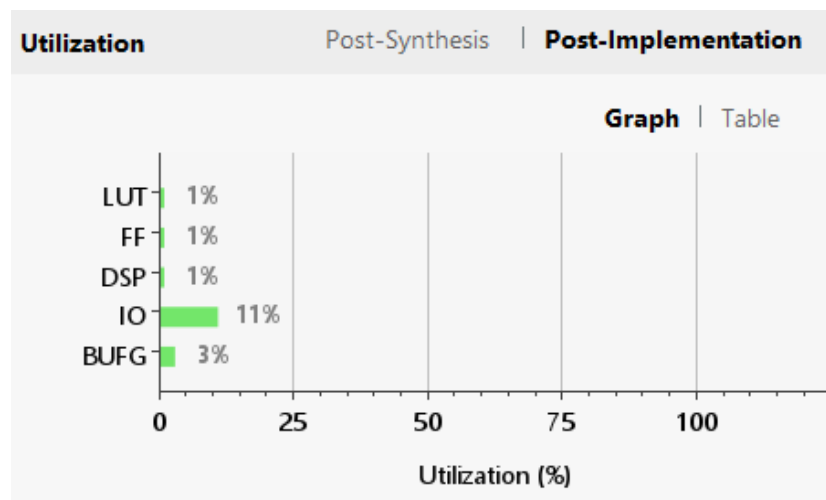


Figura 6.2: Utilizzo delle risorse FPGA dopo implementazione.

Dalla schermata di Vivado si osserva che il progetto utilizza una percentuale molto ridotta delle risorse disponibili sulla FPGA. I valori principali vengono riportati anche in Tabella 6.2.

L'utilizzo contenuto di LUT e flip-flop è coerente con la natura del progetto. Il data-path CORDIC utilizza principalmente somme, sottrazioni, shift aritmetici e registri. La LUT degli angoli contiene soltanto 16 valori e non richiede risorse di memoria significative. La presenza di una piccola percentuale di DSP è dovuta alla moltiplicazione finale utilizzata per correggere il guadagno CORDIC e ottenere il valore corretto di ρ .

Risorsa	Utilizzo indicativo
LUT	1
FF	1
DSP	1
IO	11
BUFG	3

Tabella 6.2: Percentuale indicativa di utilizzo delle risorse FPGA.

La percentuale di IO è invece legata al wrapper FPGA, che espone clock, reset, start, switch e LED.

6.3 Analisi temporale

La Figura 6.3 riporta il riepilogo temporale ottenuto dopo l'implementazione.

Design Timing Summary			
Setup		Hold	Pulse Width
Worst Negative Slack (WNS):	1,417 ns	Worst Hold Slack (WHS):	0,190 ns
Total Negative Slack (TNS):	0,000 ns	Total Hold Slack (THS):	0,000 ns
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	192	Total Number of Endpoints:	192
			86

All user specified timing constraints are met.

Figura 6.3: Timing summary del progetto dopo implementazione.

I principali risultati temporali sono riportati anche in Tabella 6.3.

Parametro	Valore
Worst Negative Slack (WNS)	1.417 ns
Total Negative Slack (TNS)	0.000 ns
Worst Hold Slack (WHS)	0.190 ns
Total Hold Slack (THS)	0.000 ns
Worst Pulse Width Slack (WPWS)	3.500 ns
Total Pulse Width Negative Slack (TPWS)	0.000 ns
Numero di endpoint falliti	0

Tabella 6.3: Risultati principali dell'analisi temporale.

Il valore positivo del Worst Negative Slack indica che il vincolo di setup è rispettato. Inoltre, il Total Negative Slack è nullo e non sono presenti endpoint falliti. Anche i vincoli di hold e pulse width risultano rispettati, come indicato dai valori positivi di WHS e WPWS.

Vivado segnala quindi che tutti i vincoli temporali specificati dall'utente sono soddisfatti. Questo conferma che l'implementazione del modulo è compatibile con il clock imposto nel progetto.

Assumendo un vincolo di clock pari a 8 ns, il valore di WNS pari a 1.417 ns indica che il ritardo del cammino critico è circa:

$$T_{\text{critico}} \approx 8 \text{ ns} - 1.417 \text{ ns} = 6.583 \text{ ns} \quad (6.2)$$

Da questo valore si può ricavare una stima indicativa della massima frequenza di funzionamento:

$$f_{\text{max}} \approx \frac{1}{6.583 \text{ ns}} \approx 151.9 \text{ MHz} \quad (6.3)$$

Questa stima conferma che il progetto presenta margine temporale rispetto al vincolo considerato.

6.4 Analisi di potenza

È stata inoltre eseguita una stima della potenza assorbita tramite il report power di Vivado. Il risultato è riportato in Figura 6.4.

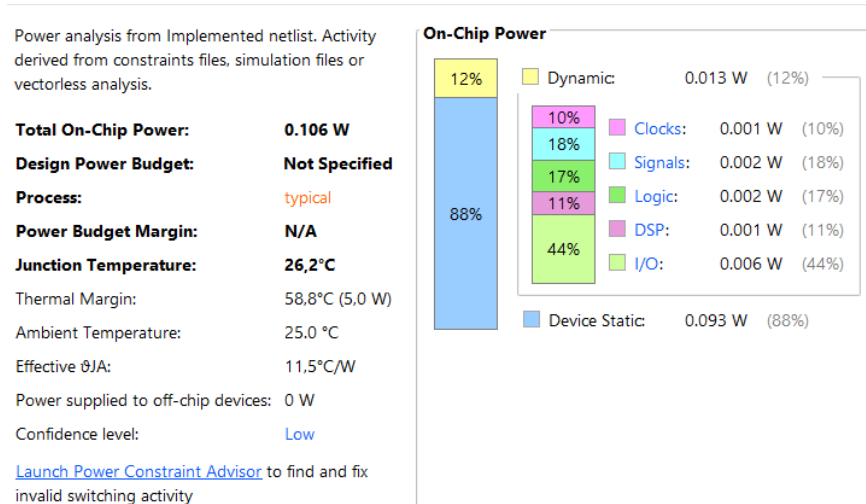


Figura 6.4: Stima della potenza assorbita ottenuta tramite il report power di Vivado.

La potenza totale stimata sul chip è pari a:

$$P_{\text{totale}} = 0.106 \text{ W} \quad (6.4)$$

La maggior parte della potenza è dovuta alla componente statica del dispositivo, mentre la componente dinamica associata all'attività del circuito è molto più bassa. I valori principali sono riportati anche in Tabella 6.4.

Componente	Potenza stimata
Potenza totale on-chip	0.106 W
Potenza dinamica	0.013 W
Potenza statica del dispositivo	0.093 W
Temperatura di giunzione	26.2°C
Margine termico	58.8°C

Tabella 6.4: Risultati principali della power analysis.

La potenza dinamica stimata è pari a 0.013 W, corrispondente a circa il 12% della potenza totale. La potenza statica del dispositivo è invece pari a 0.093 W, cioè circa l'88% della potenza totale. Questo risultato è coerente con le dimensioni ridotte del circuito implementato: il consumo dinamico del core CORDIC è limitato, mentre la componente statica dipende principalmente dal dispositivo FPGA utilizzato.

La potenza dinamica è suddivisa tra clock, segnali, logica, DSP e IO. In particolare, la componente IO risulta la più elevata tra quelle dinamiche, con un valore pari a 0.006 W. Questo è dovuto alla presenza dei segnali fisici del wrapper, come switch e LED.

Il livello di confidenza indicato da Vivado è *Low*. Di conseguenza, i valori di potenza devono essere interpretati come una stima indicativa. Una stima più accurata richiederebbe informazioni più precise sull'attività di commutazione del circuito.

6.5 Considerazioni finali sull'implementazione

I risultati ottenuti mostrano che il progetto è correttamente sintetizzabile e implementabile sul dispositivo scelto. L'utilizzo delle risorse è molto contenuto. Anche il timing risulta soddisfatto, poiché non sono presenti violazioni di setup, hold o pulse width.

La power analysis mostra una potenza totale stimata pari a 0.106 W, dominata dalla componente statica del dispositivo. La componente dinamica del circuito risulta molto bassa, coerentemente con la semplicità del datapath e con il numero limitato di segnali commutanti.

L'implementazione su FPGA conferma la correttezza strutturale del progetto RTL e mostra che il modulo CORDIC sviluppato richiede poche risorse hardware, rispetta i vincoli temporali e presenta un consumo stimato ridotto.

CAPITOLO 7

Conclusioni

In questo progetto è stato sviluppato e verificato un modulo hardware per la trasformazione da coordinate cartesiane a coordinate polari mediante algoritmo CORDIC in modalità *vectoring*. Il modulo riceve in ingresso due coordinate x e y , e produce in uscita il modulo ρ e l'angolo θ .

L'algoritmo CORDIC si è dimostrato adatto alla realizzazione hardware del sistema, poiché permette di calcolare modulo e fase di un vettore utilizzando principalmente somme, sottrazioni, shift aritmetici e una piccola tabella di lookup per i valori di $\arctan(2^{-i})$. In questo modo è stato possibile evitare l'uso diretto di operazioni più complesse, come radici quadrate e funzioni trigonometriche, che sarebbero più costose da implementare in hardware.

Il progetto RTL è stato organizzato in modo gerarchico. Il blocco principale CORDIC è stato affiancato da moduli ausiliari, tra cui la LUT degli angoli, il contatore delle iterazioni, i registri parametrizzati e il wrapper per FPGA. La macchina a stati finiti consente di gestire le fasi di attesa, caricamento, esecuzione delle iterazioni e produzione del risultato. Sono stati inoltre gestiti alcuni casi particolari, come il vettore nullo e i vettori appartenenti al semipiano sinistro, per i quali è stata introdotta una correzione di quadrante.

La verifica funzionale è stata effettuata tramite simulazione in ModelSim. Il testbench ha permesso di controllare il comportamento del modulo in diversi casi significativi, comprendenti vettori nei quattro quadranti, il vettore nullo, casi generici e un impulso di `start` applicato durante l'elaborazione. I risultati ottenuti hanno confermato la correttezza del funzionamento del circuito: i valori di ρ e θ risultano coerenti con quelli teorici, con scostamenti limitati dovuti alla quantizzazione fixed-point e al numero finito di iterazioni.

L'implementazione su FPGA tramite Vivado ha confermato la sintetizzabilità del progetto. Il dispositivo utilizzato appartiene alla famiglia Zynq-7000 e il modulo di livello superiore scelto per l'implementazione è stato `CORDIC_WRAPPER`. I report di implementazione hanno mostrato un utilizzo molto contenuto delle risorse hardware, con percentuali ridotte per LUT, flip-flop e DSP. Anche l'analisi temporale ha dato esito positivo, senza violazioni di setup, hold o pulse width. La stima di potenza ha inoltre evidenziato un consumo complessivo ridotto, dominato principalmente dalla componente statica del dispositivo FPGA.

Nel complesso, il progetto ha raggiunto gli obiettivi prefissati. È stato realizzato un modulo CORDIC funzionante, verificato mediante simulazione e implementabile su FPGA. Il sistema risulta compatto dal punto di vista delle risorse utilizzate, rispetta i vincoli temporali e produce risultati numericamente coerenti con la trasformazione cartesiano-polare attesa.

Possibili sviluppi futuri potrebbero riguardare l'aumento del numero di iterazioni per migliorare ulteriormente la precisione, l'introduzione di un'interfaccia di ingresso più flessibile per testare vettori arbitrari direttamente su FPGA, oppure l'estensione del wrapper per visualizzare l'intero risultato su un'interfaccia più completa.